

Machine Learning Zoomcamp

Module 2 — Regression: Car Price Prediction

Study notes with equations and charts

Based on the DataTalksClub course notes, 02-regression

Condensed and re-organised from

<https://github.com/DataTalksClub/machine-learning-zoomcamp/tree/main/02-regression>.

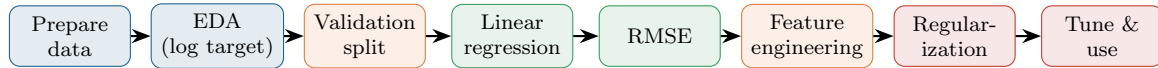
All charts are computed on the real course dataset (data.csv, 11 914 cars, seed 2). Numbers can differ from the videos in the 3rd decimal depending on NumPy/pandas versions.

Contents

1 The Project	2
2 Data Preparation	2
3 Exploratory Data Analysis	2
3.1 The target has a long tail	2
3.2 Missing values	2
4 Validation Framework	3
5 Linear Regression	3
5.1 One observation	3
5.2 Vector form	4
6 Training: the Normal Equation	4
7 Baseline Model	5
8 Root Mean Squared Error	5
9 Validation RMSE and prepare_X	6
10 Feature Engineering and Categorical Variables	6
10.1 Age	6
10.2 One-hot encoding	6
11 Regularization	7
11.1 Why the model blew up	7
11.2 Ridge regression	7
12 Tuning the Model	8
13 Using the Model	8
14 Summary	9
A Appendix: The Normal Equation, Step by Step	9
A.1 Why there is no exact solution	9
A.2 What multiplying by $\mathbf{X}^T$ does	10
A.3 Why it is the best answer	10
A.4 Worked example: a line through three points	10
A.5 When $\mathbf{X}^T \mathbf{X}$ has no inverse	10

1 The Project

A user sells a car on a classifieds site and must enter a price. We train a model that suggests one from the car's characteristics. Data: a Kaggle dataset of cars with make, model, year, engine, fuel type, ... and the target **MSRP** (Manufacturer Suggested Retail Price).



The module implements everything from scratch with NumPy/pandas; from Module 3 on the same ideas come from scikit-learn.

2 Data Preparation

Make column names and string values uniform: lowercase, spaces $\rightarrow$ underscores.

```
df = pd.read_csv('data.csv')
df.columns = df.columns.str.lower().str.replace(' ', '_')
strings = list(df.dtypes[df.dtypes == 'object'].index)
for col in strings:
    df[col] = df[col].str.lower().str.replace(' ', '_')
```

pandas 3.x gotcha

Since pandas 3.0 text columns get the dtype `str`, not `object`, so `df.dtypes == 'object'` returns *no* columns and the strings are silently left un-normalised. Portable version:

```
strings = list(df.select_dtypes(include=['object', 'string']).columns)
```

3 Exploratory Data Analysis

Looping over columns with `unique()` and `nunique()` shows e.g. 48 makes, 914 models, 28 years, 10 fuel types.

3.1 The target has a long tail

Most cars are cheap, a few cost over \$1M. Such *long-tail* distributions confuse linear models, so we model the **log** of the price:

$$y_i = \log(1 + \text{msrp}_i) \quad (\text{np.log1p}), \quad \text{msrp}_i = e^{y_i} - 1 \quad (\text{np.expm1}).$$

The “+1” guards against $\log 0 = -\infty$. The logarithm compresses large values: $\log(1 + 10^3) \approx 6.9$, $\log(1 + 10^5) \approx 11.5$.

3.2 Missing values

`df.isnull().sum()` gives: `engine_fuel_type` 3, `engine_hp` 69, `engine_cylinders` 30, `number_of_doors` 6, `market_category` 3742. They must be handled before training (see §7).

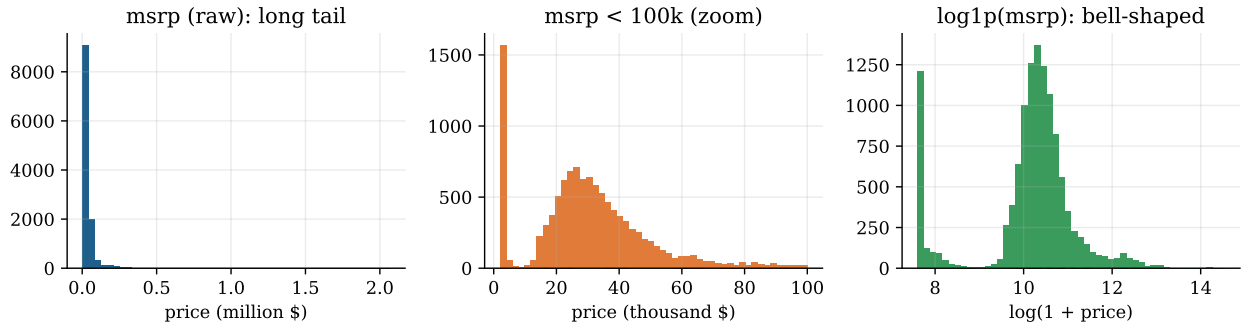


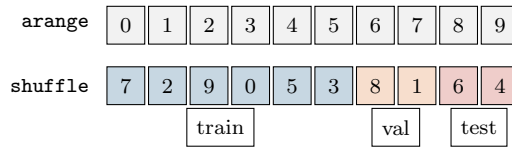
Figure 1: Price distribution: raw (long tail), zoomed below \$100k (note the spike at \$1,000, probably the platform minimum), and after $\log_{1p}$: close to a normal (bell) shape.

4 Validation Framework

Split 60/20/20. Compute validation and test sizes first and give the remainder to train, so rounding never drops rows:

$$n_{\text{val}} = n_{\text{test}} = \lfloor 0.2n \rfloor = 2382, \quad n_{\text{train}} = n - n_{\text{val}} - n_{\text{test}} = 11914 - 4764 = 7150.$$

The file is sorted by make, so a sequential cut would put all BMWs in validation. **Shuffle** the row indices first:



```
np.random.seed(2) # reproducible shuffle
idx = np.arange(n); np.random.shuffle(idx)
df_train = df.iloc[idx[:n_train]].reset_index(drop=True)
df_val = df.iloc[idx[n_train:n_train+n_val]].reset_index(drop=True)
df_test = df.iloc[idx[n_train+n_val:]].reset_index(drop=True)
y_train = np.log1p(df_train.msrp.values) # same for val / test
del df_train['msrp'] # same for val / test
```

Target leakage

Always delete the target from the feature DataFrames. If `msrp` slips into **X**, the model “predicts the price from the price” and looks perfect.

5 Linear Regression

5.1 One observation

For car i with features $\mathbf{x}_i = (x_{i1}, \dots, x_{in})$:

$$g(\mathbf{x}_i) = w_0 + \sum_{j=1}^n w_j x_{ij}$$

w_0 is the **bias** (the prediction knowing nothing about the car); w_j is how much one unit of feature j moves the (log) price.

Example (Rolls-Royce Phantom, hand-picked weights): features horsepower 453, city mpg 11, popularity 86; $w_0 = 7.17$, $\mathbf{w} = (0.01, 0.04, 0.002)$:

$$g(\mathbf{x}_{10}) = 7.17 + 453 \cdot 0.01 + 11 \cdot 0.04 + 86 \cdot 0.002 = 7.17 + 4.53 + 0.44 + 0.172 = 12.312$$

$$\text{price} = \text{expm1}(12.312) \approx \$222\,347.$$

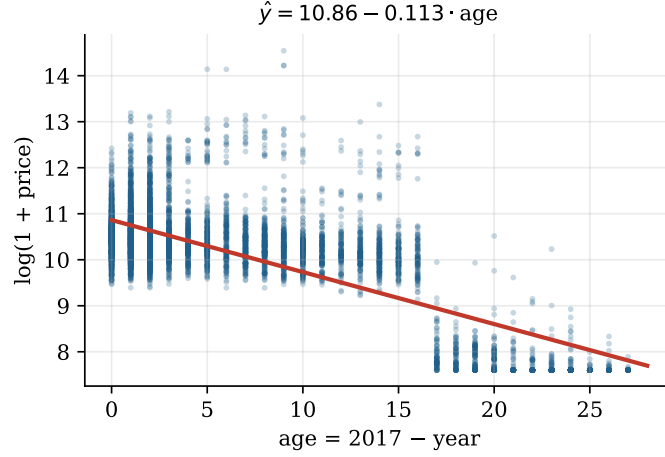


Figure 2: Linear regression with a single feature on the training data: every extra year of age lowers the log price by ≈ 0.113 , i.e. multiplies the price by $e^{-0.113} \approx 0.89$ ($\approx 11\%$ per year).

5.2 Vector form

The sum is a dot product. Adding a constant “fictional” feature $x_{i0} = 1$ absorbs the bias:

$$g(\mathbf{x}_i) = w_0 + \mathbf{x}_i^T \mathbf{w} = \tilde{\mathbf{x}}_i^T \tilde{\mathbf{w}}, \quad \tilde{\mathbf{x}}_i = (1, x_{i1}, \dots, x_{in}), \quad \tilde{\mathbf{w}} = (w_0, w_1, \dots, w_n).$$

For all m cars at once it is a matrix–vector product:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = \begin{pmatrix} 1 & 148 & 24 & 1385 \\ 1 & 132 & 25 & 2031 \\ 1 & 453 & 11 & 86 \end{pmatrix} \begin{pmatrix} 7.17 \\ 0.01 \\ 0.04 \\ 0.002 \end{pmatrix} = \begin{pmatrix} 12.380 \\ 13.552 \\ 12.312 \end{pmatrix}.$$

Evolution in code: a for loop $\rightarrow \text{dot}(\mathbf{x}_i, \mathbf{w}) \rightarrow \text{prepend } 1 \rightarrow \mathbf{X}.\text{dot}(\mathbf{w})$.

6 Training: the Normal Equation

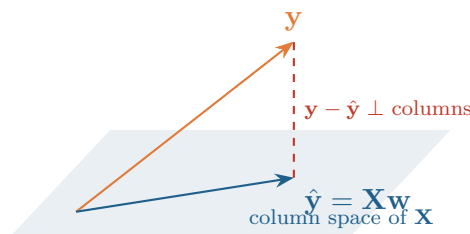
We want $\mathbf{X}\mathbf{w} \approx \mathbf{y}$ with $\mathbf{X} \in \mathbb{R}^{m \times (n+1)}$, $m \gg n$. $\mathbf{X}$ is not square, so $\mathbf{X}^{-1}$ does not exist and the system has no exact solution. Multiply both sides by $\mathbf{X}^T$:

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y} \implies \boxed{\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}}$$

$\mathbf{X}^T \mathbf{X}$ is the $(n+1) \times (n+1)$ **Gram matrix**; being square it usually has an inverse. (A step-by-step explanation with a worked example is in [Appendix A](#).)

Why it is the “closest” solution

The normal equation minimises the squared error $L(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \sum_i (\mathbf{x}_i^T \mathbf{w} - y_i)^2$. Setting the gradient to zero, $\nabla L = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0}$, gives exactly $\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$. Geometrically, $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ is the orthogonal projection of $\mathbf{y}$ onto the space spanned by the columns of $\mathbf{X}$: the residual $\mathbf{y} - \hat{\mathbf{y}}$ is perpendicular to every column.



```
def train_linear_regression(X, y):
    ones = np.ones(X.shape[0])
    X = np.column_stack([ones, X]) # bias column
    XTX = X.T.dot(X) # Gram matrix
    XTX_inv = np.linalg.inv(XTX)
    w_full = XTX_inv.dot(X.T).dot(y)
    return w_full[0], w_full[1:] # w0, w
```

7 Baseline Model

Five numerical features: `engine_hp`, `engine_cylinders`, `highway_mpg`, `city_mpg`, `popularity`. Training straight away returns `nan` weights because of missing values. Simplest fix: `fillna(0)`. Why is zero acceptable? A zero feature simply drops out of the sum:

$$g(\mathbf{x}_i) = w_0 + \underbrace{0 \cdot w_1}_{=0} + x_{i2}w_2 = w_0 + x_{i2}w_2,$$

so the model effectively ignores the missing feature (filling with the mean is often more sensible). Result: $w_0 \approx 7.93$.

8 Root Mean Squared Error

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (g(\mathbf{x}_i) - y_i)^2}$$

Squaring prevents positive/negative errors from cancelling and penalises big errors more; the root brings the value back to the units of y .

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	
$\hat{y}_i$	10	9	11	10	
y_i	9	9	10.5	11.5	$\Rightarrow \text{MSE} = \frac{3.5}{4} = 0.875, \text{ RMSE} = \sqrt{0.875} \approx 0.935$
$\hat{y}_i - y_i$	1	0	0.5	-1.5	
$(\hat{y}_i - y_i)^2$	1	0	0.25	2.25	

```
def rmse(y, y_pred):
    se = (y - y_pred) ** 2
    return np.sqrt(se.mean())
```

Reading RMSE on a log target: an error δ in $\log(1 + \text{price})$ is roughly a *multiplicative* error e^δ in price. RMSE ≈ 0.46 means typical predictions are off by a factor of about $e^{0.46} \approx 1.6$, i.e. tens of percent, not dollars.

9 Validation RMSE and prepare_X

The baseline scores RMSE 0.755 on *train*; the honest number is on *validation*: **0.762**. A single `prepare_X(df)` guarantees that train, validation and test go through identical transformations:

```
def prepare_X(df):
    df = df.copy() # never modify the caller's data
    features = base.copy() # never mutate the global list
    df['age'] = 2017 - df.year
    features.append('age')
    # ... more features ...
    return df[features].fillna(0).values

X_train = prepare_X(df_train); w0, w = train_linear_regression(X_train, y_train)
X_val = prepare_X(df_val); y_pred = w0 + X_val.dot(w); rmse(y_val, y_pred)
```

10 Feature Engineering and Categorical Variables

10.1 Age

The data was collected in 2017, so $\text{age}_i = 2017 - \text{year}_i$. One feature brings validation RMSE from 0.762 to **0.517**.

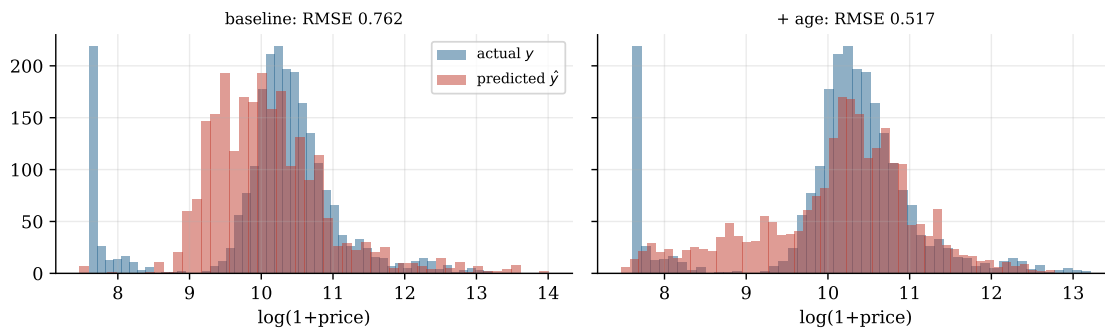


Figure 3: Validation set: predicted vs actual log prices. The baseline is shifted left (under-predicts); with `age` the two distributions almost overlap.

10.2 One-hot encoding

A categorical variable c with values $\{v_1, \dots, v_K\}$ becomes K binary columns:

$$x_{i,c=v} = \mathbf{1}[c_i = v] \in \{0, 1\}, \quad \sum_{k=1}^K x_{i,c=v_k} = 1.$$

number_of_doors	doors_2	doors_3	doors_4
2	1	0	0
3	0	1	0
4	0	0	1

`number_of_doors` looks numeric but is really a category. The course keeps the top-5 values of each categorical column (`value_counts().head()`); `model` is skipped (914 values).

```
for c, values in categories.items():
    for v in values:
        df['%s_%s' % (c, v)] = (df[c] == v).astype('int')
        features.append('%s_%s' % (c, v))
```

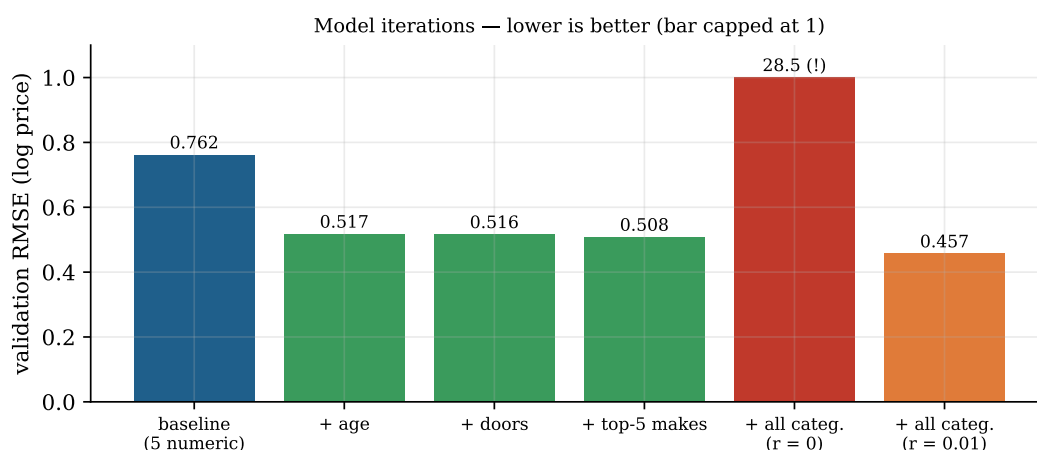


Figure 4: Validation RMSE through the iterations. Doors barely help, makes help $\sim 1\%$, and adding all categories *without* regularization explodes (weights up to $\sim 10^{15}$). A small r fixes it and gives the best model.

11 Regularization

11.1 Why the model blew up

$(\mathbf{X}^T \mathbf{X})^{-1}$ does not exist when a column of $\mathbf{X}$ is a linear combination of others (e.g. duplicates, or one-hot columns that always co-occur). With real, slightly noisy data it becomes *nearly* singular: NumPy returns an inverse, but with enormous entries, hence enormous weights.

$$A = \begin{pmatrix} 1 & 2 & 2 \\ 2 & 1 & 1.0000001 \\ 2 & 1.0000001 & 1 \end{pmatrix} : \quad \max |A^{-1}| \approx 5 \times 10^6; \quad \max |(A + 0.01I)^{-1}| \approx 50.$$

11.2 Ridge regression

Add a small r to the diagonal of the Gram matrix:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X} + rI)^{-1} \mathbf{X}^T \mathbf{y}$$

This is the solution of the penalised problem $\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + r \|\mathbf{w}\|^2$: large weights now cost something, so they stay small. $r = 0$ is plain linear regression; larger r gives smaller weights (and eventually underfitting).

```
def train_linear_regression_reg(X, y, r=0.001):
    X = np.column_stack([np.ones(X.shape[0]), X])
    XTX = X.T.dot(X) + r * np.eye(X.shape[1]) # the only new line
    w_full = np.linalg.inv(XTX).dot(X.T).dot(y)
    return w_full[0], w_full[1:]
```

12 Tuning the Model

r is a **hyperparameter**: we pick it on the validation set.

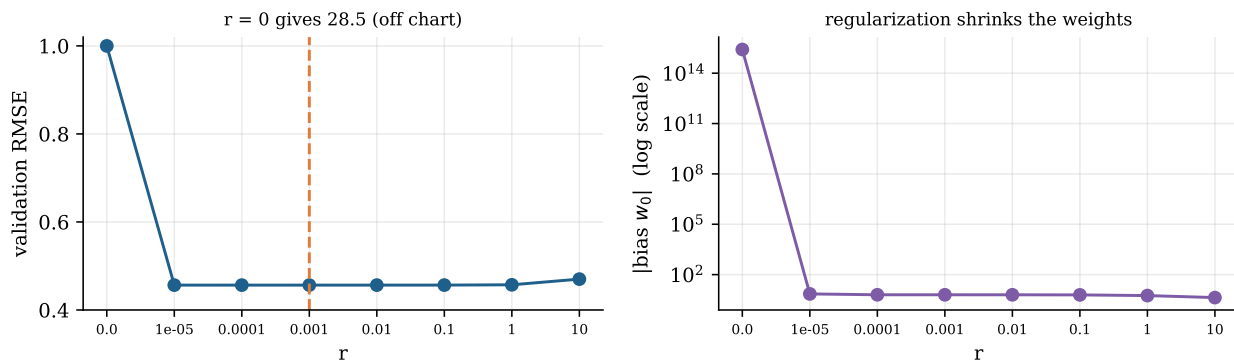


Figure 5: Left: validation RMSE vs r . Any $r > 0$ removes the instability; the curve is flat until $r \approx 1$ and degrades at $r = 10$. Right: the bias term shrinks as r grows. The course picks $r = 0.001$.

13 Using the Model

With r fixed, retrain on **train + validation** (more data) and check once on test:

```
df_full_train = pd.concat([df_train, df_val]).reset_index(drop=True)
y_full_train = np.concatenate([y_train, y_val])
w0, w = train_linear_regression_reg(prepare_X(df_full_train), y_full_train, r=0.001)
y_pred = w0 + prepare_X(df_test).dot(w); rmse(y_test, y_pred) # ~0.45-0.46
```

Test RMSE $\approx$ validation RMSE $\Rightarrow$ the model generalises, it did not win by luck.

Predicting one car. In production a request arrives as a dictionary; wrap it in a one-row DataFrame:

```
car = df_test.iloc[20].to_dict() # a Toyota Sienna, 2015
X_small = prepare_X(pd.DataFrame([car]))
np.exp1(w0 + X_small.dot(w)[0]) # predicted price in $
```

The actual price is \$35 000; the course reports a prediction of $\approx$ \$41 500 (my re-run gives $\approx$ \$35 000; small differences come from library versions).

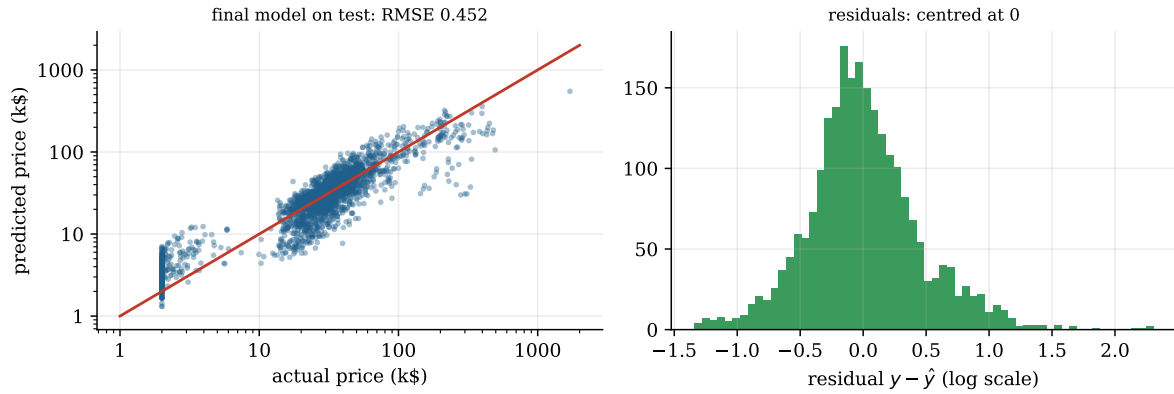


Figure 6: Final model on the test set. Left: predicted vs actual price (log axes; red = perfect prediction). Right: residuals $y - \hat{y}$ on the log scale, centred on zero.

14 Summary

Preparation & EDA	Normalise names/strings; long-tailed target $\Rightarrow y = \log(1 + \text{price})$; find missing values.
Validation	Shuffle (seed), 60/20/20, reset index, take y out and delete it from the features.
Linear regression	$g(\mathbf{x}_i) = w_0 + \mathbf{x}_i^T \mathbf{w}$; for all rows $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ with a column of ones.
Normal equation	$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ minimises $\ \mathbf{X}\mathbf{w} - \mathbf{y}\ ^2$ — “not magic, just math”.
RMSE	$\sqrt{\frac{1}{m} \sum (\hat{y}_i - y_i)^2}$ on validation; one number to compare models.
Features	prepare <code>X</code> for identical processing; <code>age</code> ; one-hot top-5 categories.
Regularization	$\mathbf{w} = (\mathbf{X}^T \mathbf{X} + rI)^{-1} \mathbf{X}^T \mathbf{y}$ tames near-singular Gram matrices.
Final model	Tune r on validation; retrain on train+val; confirm on test; <code>expm1</code> to get dollars.

RMSE journey (validation): 0.762 (baseline) $\rightarrow$ 0.517 (+age) $\rightarrow$ 0.516 (+doors) $\rightarrow$ 0.508 (+makes) $\rightarrow$ 28–41 (all categories, $r = 0$) $\rightarrow$ ≈ 0.46 (regularised).

Explore more: try the top-10 values per category instead of top-5; practise on the California housing or UCI Student Performance datasets.

A Appendix: The Normal Equation, Step by Step

A.1 Why there is no exact solution

Read $\mathbf{X}\mathbf{w} \approx \mathbf{y}$ through the shapes: $\underbrace{\mathbf{X}}_{m \times (n+1)} \underbrace{\mathbf{w}}_{(n+1) \times 1} \approx \underbrace{\mathbf{y}}_{m \times 1}$. It is a system of m equations (one per car, ≈ 7150) in $n + 1$ unknowns (the weights). With far more equations than unknowns the system is **overdetermined**: no single $\mathbf{w}$ reproduces every noisy price exactly, just as a straight line cannot pass through three points that are not aligned. And $\mathbf{w} = \mathbf{X}^{-1}\mathbf{y}$ is impossible because only square matrices have inverses.

A.2 What multiplying by $\mathbf{X}^T$ does

$$\underbrace{\mathbf{X}^T \mathbf{X}}_{(n+1) \times (n+1)} \mathbf{w} = \underbrace{\mathbf{X}^T \mathbf{y}}_{(n+1) \times 1}$$

The system shrinks from m equations to $n + 1$ equations in $n + 1$ unknowns: it is now *square* and (usually) solvable. This is not just a trick to fix shapes — the solution of this smaller system is the best approximate solution of the original one.

A.3 Why it is the best answer

“Best” means smallest sum of squared errors:

$$L(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \sum_{i=1}^m (\mathbf{x}_i^T \mathbf{w} - y_i)^2.$$

L is a bowl-shaped (convex quadratic) function of $\mathbf{w}$; its minimum is where the gradient vanishes:

$$\nabla L = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0} \iff \mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}.$$

So the normal equation *is* the minimum condition of the squared error — which is also why RMSE is the natural metric for this model.

Geometric meaning. The condition reads $\mathbf{X}^T(\mathbf{y} - \hat{\mathbf{y}}) = \mathbf{0}$: the residual vector is orthogonal (“normal”) to every column of $\mathbf{X}$. Whatever error remains carries no information that any feature could still explain — otherwise adjusting that feature’s weight would reduce the error.

A.4 Worked example: a line through three points

Fit $\hat{y} = w_0 + w_1 x$ to $(1, 1), (2, 2), (3, 2)$:

$$\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 1 \\ 2 \\ 2 \end{pmatrix}, \quad \mathbf{X}^T \mathbf{X} = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix} = \begin{pmatrix} m & \sum x_i \\ \sum x_i & \sum x_i^2 \end{pmatrix}, \quad \mathbf{X}^T \mathbf{y} = \begin{pmatrix} 5 \\ 11 \end{pmatrix} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \end{pmatrix}.$$

With $\det = 3 \cdot 14 - 6 \cdot 6 = 6$:

$$(\mathbf{X}^T \mathbf{X})^{-1} = \frac{1}{6} \begin{pmatrix} 14 & -6 \\ -6 & 3 \end{pmatrix}, \quad \mathbf{w} = \frac{1}{6} \begin{pmatrix} 14 \cdot 5 - 6 \cdot 11 \\ -6 \cdot 5 + 3 \cdot 11 \end{pmatrix} = \begin{pmatrix} 0.667 \\ 0.5 \end{pmatrix}.$$

Predictions $\hat{\mathbf{y}} = (1.167, 1.667, 2.167)$, residuals $\mathbf{y} - \hat{\mathbf{y}} = (-0.167, 0.333, -0.167)$. Orthogonality check:

$$\mathbf{1}^T(\mathbf{y} - \hat{\mathbf{y}}) = -0.167 + 0.333 - 0.167 = 0, \quad \mathbf{x}^T(\mathbf{y} - \hat{\mathbf{y}}) = -0.167 + 0.667 - 0.5 = 0.$$

A.5 When $\mathbf{X}^T \mathbf{X}$ has no inverse

If a column of $\mathbf{X}$ is a linear combination of others (a duplicated feature, the same quantity in two units, one-hot columns that always co-occur), infinitely many $\mathbf{w}$ give the same predictions and $\mathbf{X}^T \mathbf{X}$ is singular. With real, slightly noisy data it is *nearly* singular: NumPy still inverts it, but returns huge numbers — the RMSE explosion of §11. Ridge regression fixes it by adding rI before inverting: $\mathbf{w} = (\mathbf{X}^T \mathbf{X} + rI)^{-1} \mathbf{X}^T \mathbf{y}$.

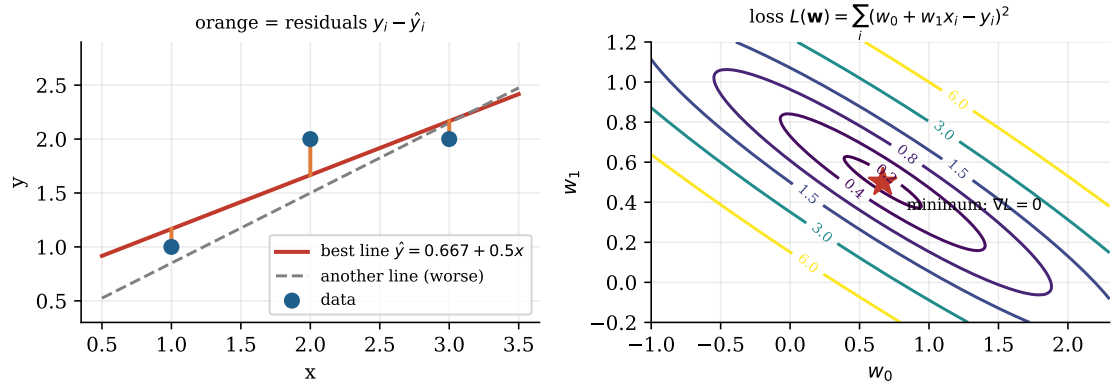


Figure 7: Left: the least-squares line and its residuals; any other line has a larger sum of squared residuals. Right: the loss surface over (w_0, w_1) is a bowl; the normal equation lands exactly at its bottom.